

Student name: Jordan Elkin

Student ID:226621873

SIT225: Data Capture Technologies

Activity 2.1: Working with sensor - DHT22

DHT22 is a temperature and humidity sensor.

Hardware Required

Arduino Board

DHT22 sensor

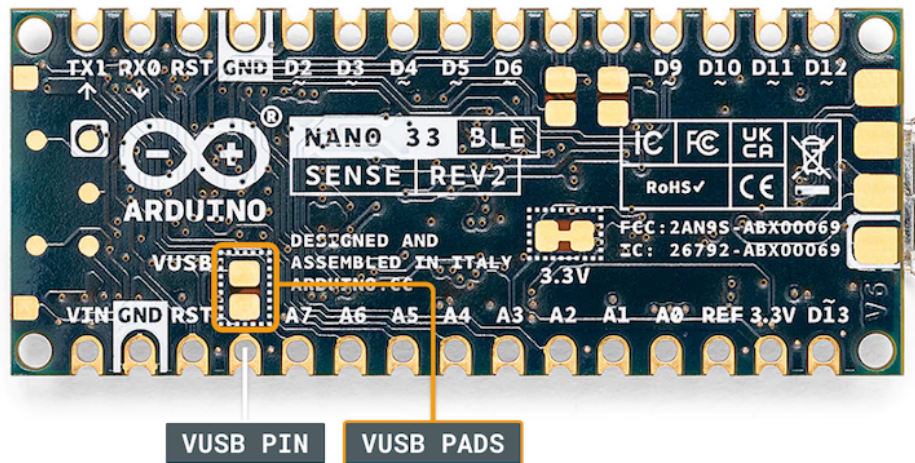
USB cable

Software Required

Arduino programming environment

Known issue, action required

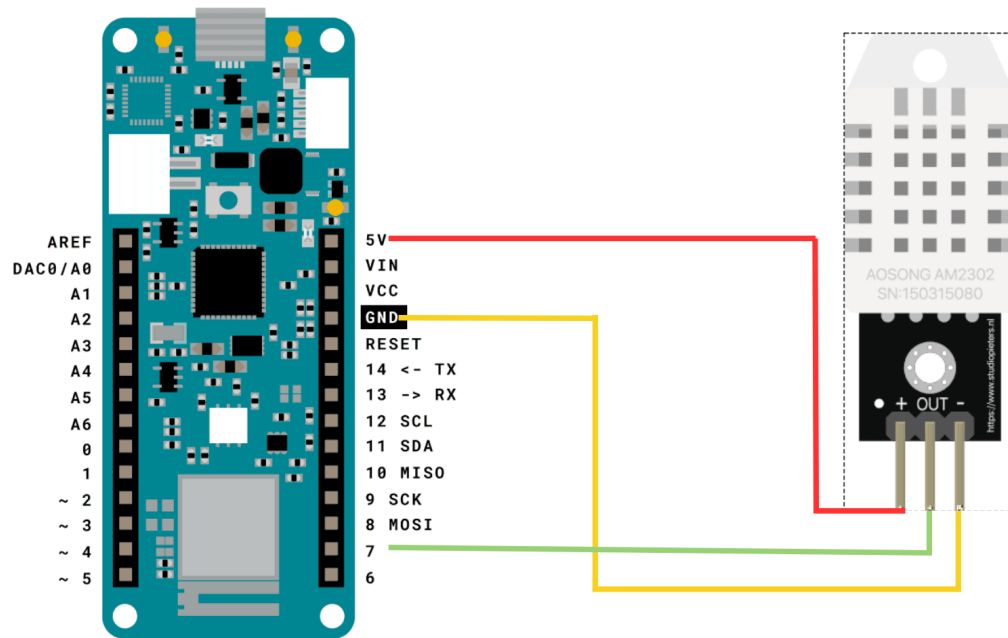
Arduino Nano 33 IoT board operates on 3.3 V, it needs to be arranged to make it 5 V. The Arduino board has a pin called VUSB or VBUS and there are 2 pads next to the pin. **These two pads must be shorted to enable the pin** (see detail here <https://support.arduino.cc/hc/en-us/articles/360014779679-Enable-5-V-power-on-the-VUSB-or-VBUS-pin-on-Nano-boards>).



To test, you can use a wire and connect these 2 pads manually by hand to see if data is coming through DHT22 sensor. For long data collection, you will need to solder the wire permanently to connect the pads. Seeking help from tutors there is an on-campus facility called Maker Space where you can do it.

Steps:

Step	Action
1	Connect your DHT22 Temperature and Humidity Sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.



- Pick a red male-female jumper wire and attach the female end to pin 1 (VCC pin) on the sensor. Plug the male end into the Arduino board's 5V power pin.
- Pick a blue male-female jumper wire and attach the female end to pin 2 (DATA pin) on the sensor. Plug the male end into the Arduino board's digital data pin 2.
- Pick a black male-female jumper wire and attach the female end to pin 4 (GND) on the sensor. Plug the male end into the Arduino board's GND pin.
- Sensor's pin 3 is not used.

2 Connect your Arduino board to your computer using the USB cable.

3 Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_dht22.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

```

6
7 // Add "DHT sensor library"
8 #include <DHT.h>
9
10 #define DHTPIN 2 // digital pin number
11 #define DHTTYPE DHT22 // DHT type 11 or 22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // variables to store data.
15 float hum, temp;
16
17 void setup() {
18     // Set baud rate for serial communication
19     Serial.begin(9600);
20
21     // initialise DHT library
22     dht.begin();
23 }
24
25 void loop() {
26     // read data
27     hum = dht.readHumidity();
28     temp = dht.readTemperature();
29
30     // Print data to serial port - a long way
31     //
32     // Serial.print("Humid: ");
33     // Serial.print(hum);
34     // Serial.print(" %, Temp: ");
35     // Serial.print(temp);
36     // Serial.println(" Celsius");
37
38     // Print data to serial port - a compact way
39     Serial.println(String(hum) + "," + String(temp));
40
41     // wait a while
42     delay(15*1000);
43 }

```

4 **Question:** A spec of the DHT22 sensor is given in the link below. It mentions that the sampling rate is 0.5 Hz.

<https://lastminuteengineers.com/dht11-dht22-arduino-tutorial>

- i) What does the sampling rate mean?
- ii) Where is this used in the Arduino code?

Answer:

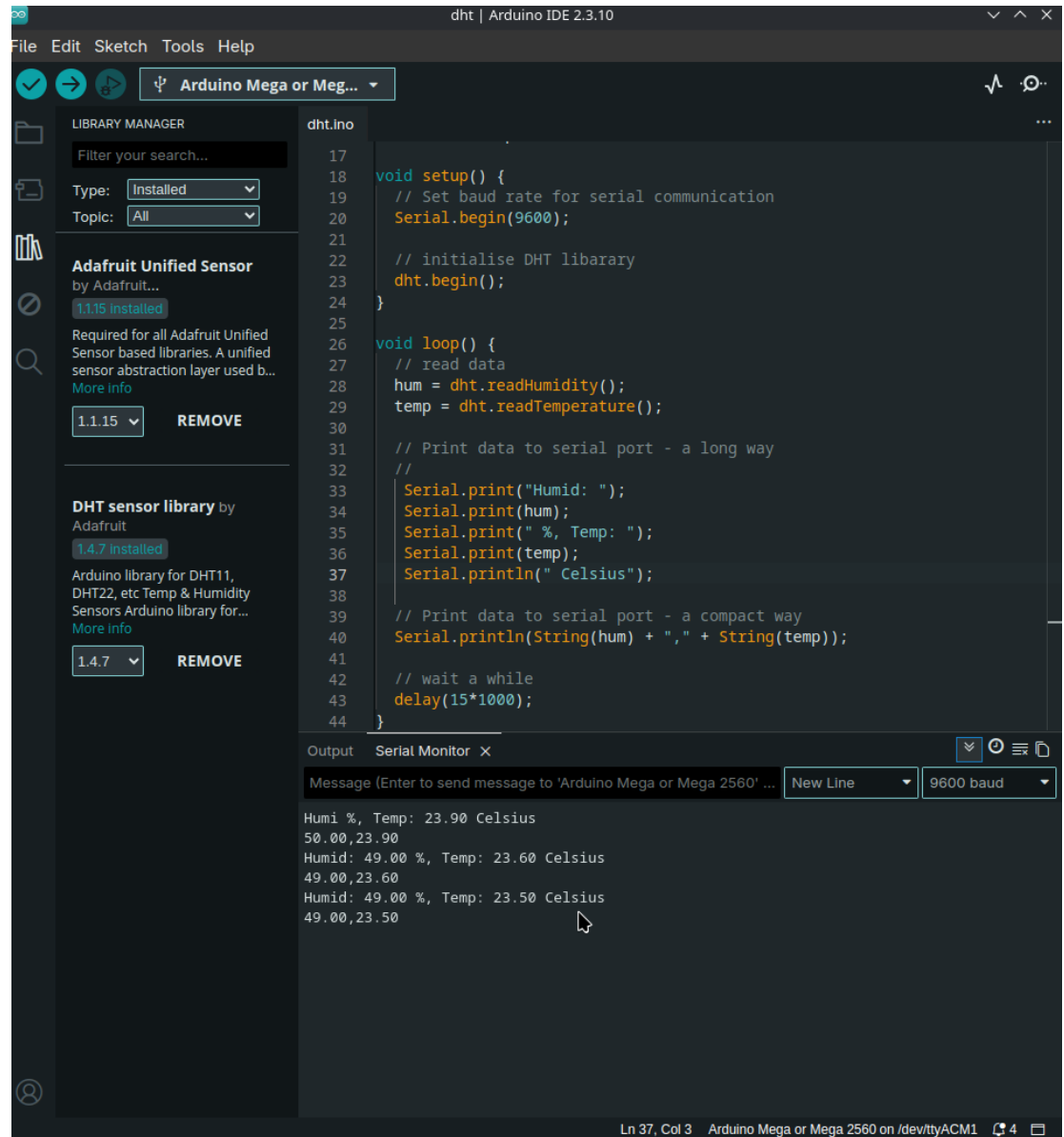
i) The sampling rate means the amount of times per second that the Arduino will receive input data from the sensor. This means that the sensor will only output new temperature and humidity readings every 0.5 seconds and if the sensor is polled before this time it will not have data available..

ii) Sampling in the above code is done in the method readHumidity() and readTemperature(). The standoff time is the delay which means

that the loop repeats when the sensor is ready to output new data.

- 5 **Question:** Take a screenshot of your Serial Monitor displaying temperature & humidity sensor data logs. Add the image here.

Answer:



Activity 2.2: Working with sensor - HC-SR04

HC-SR04 is an Ultrasonic sensor.

Hardware Required

Arduino Board

HC-SR04 Ultrasonic sensor

USB cable

Software Required

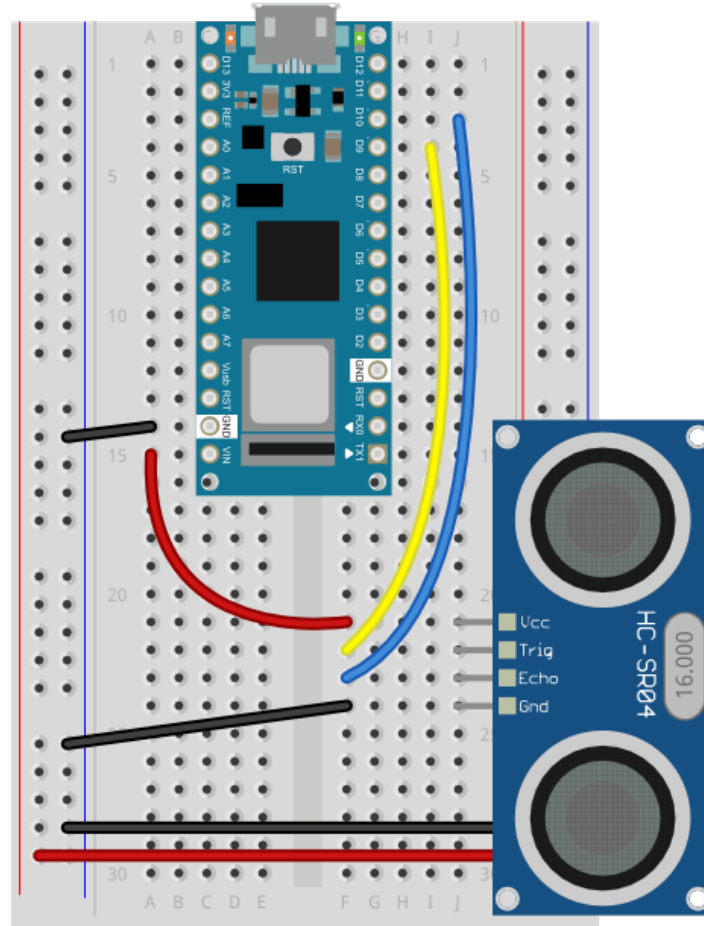
Arduino programming environment

Known issue, action required

The same known issue applies to SR04 which operates at 5 V source while the Arduino Nano 33 IoT supplies 3.3 V. It requires 2 VUSB (or VBUS) pads to be shorted to enable the pin.

Steps:

Step	Action
1	Connect your HC-SR04 sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.



2 Connect your Arduino board to your computer using the USB cable.

3 Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_hcsr04_distance.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

```

7  const int trigger = 2;
8  const int echo = 3;
9
10 int getUltrasonicDistance(){
11     // Function to retrieve the distance reading of the ultrasonic
12     long duration;
13     int distance;
14
15     // Assure the trigger pin is LOW:
16     digitalWrite(trigger, LOW);
17     // Brief pause:
18     delayMicroseconds(5);
19
20     // Trigger the sensor by setting the trigger to HIGH:
21     digitalWrite(trigger, HIGH);
22     // Wait a moment before turning off the trigger:
23     delayMicroseconds(10);
24     // Turn off the trigger:
25     digitalWrite(trigger, LOW);
26
27     // Read the echo pin:
28     duration = pulseIn(echo, HIGH);
29     // Calculate the distance in centimeter (CM):
30     distance = duration * 0.034 / 2;
31
32     // Uncomment this line to return value in IN instead of CM:
33     //distance = distance * 0.3937008
34
35     // Return the distance read from the sensor:
36     return distance;
37 }
38
39 void setup() {
40     // Define inputs and outputs:
41     pinMode(trigger, OUTPUT);
42     pinMode(echo, INPUT);
43
44     // Start the serial monitor:
45     Serial.begin(9600);
46 }
47
48 void loop() {
49     // Print the distance to the serial monitor:
50     Serial.print("Distance: ");
51     Serial.println(getUltrasonicDistance());
52
53     // Wait one second before continuing:
54     delay(1000);
55 }

```

4

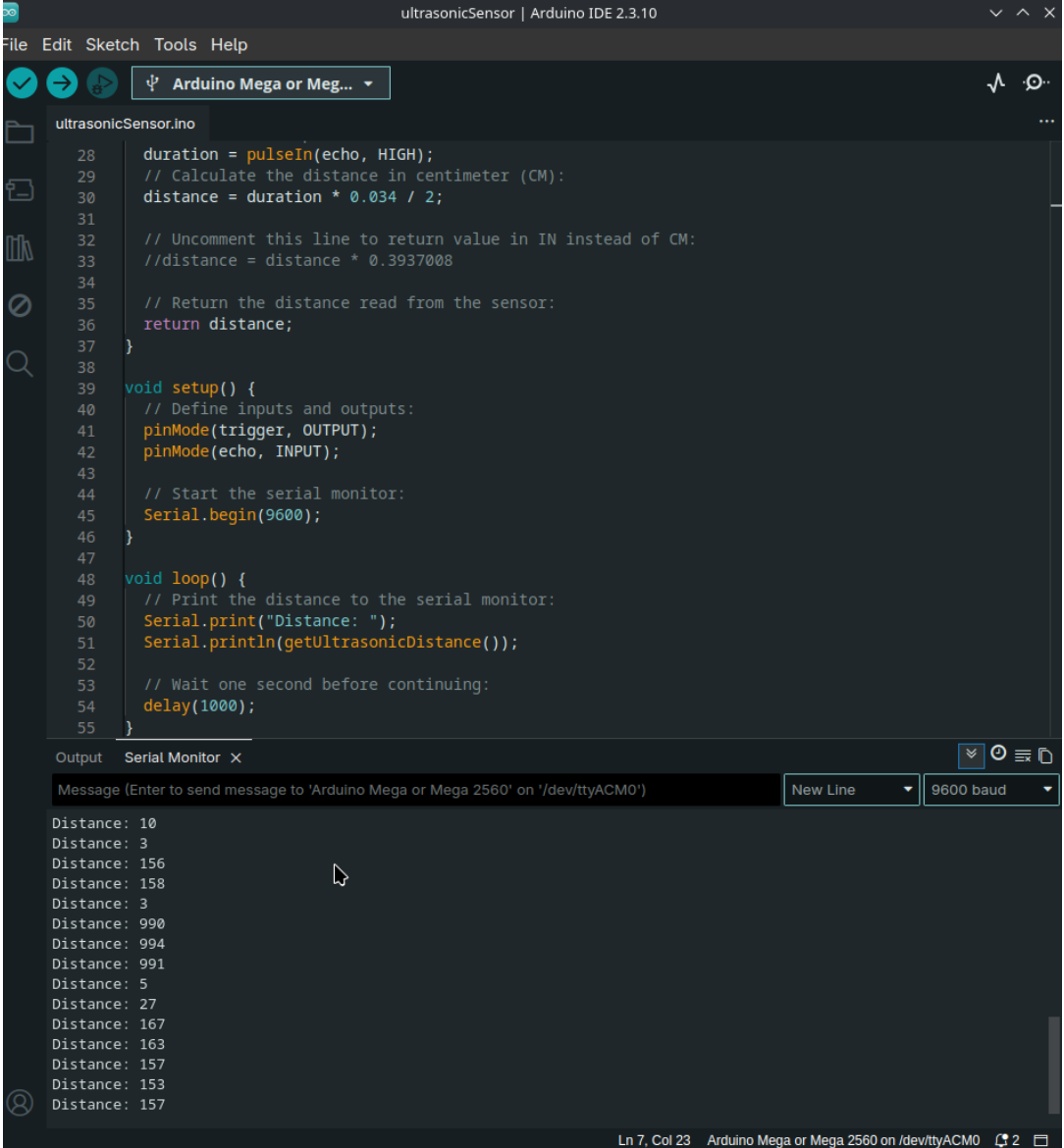
Question: Spec of SR04 is available here (<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>). Identify 2 critical aspects you should be careful about this sensor operation.

Answer:

1)The sensor operates at 5v DC and has a maximum working current of 15mA. Exceeding these will damage the device.

2) The module automatically send eight 40khz pulses to detect whether there is a signal back. If the signal does come back the distance needs to be halved to measure the distance an object is from the sensor not there and back. Further, the trigger input pulse is 10 microsecond pulse.

- 5 **Question:** Take a screenshot of your Serial Monitor displaying distance values while you try to generate a periodic motion by moving your hand gradually back and forth towards the sensor. Add the image here.



The screenshot displays the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. Below the menu is a toolbar with icons for checking, running, and uploading code, along with a dropdown menu for the board type, currently set to 'Arduino Mega or Meg...'. The main editor window shows the code for 'ultrasonicSensor.ino'. The code defines a function to calculate distance from an ultrasonic sensor, with comments in Italian. It includes a setup function to initialize pins and the serial monitor, and a loop function to continuously read and print the distance. The Serial Monitor window at the bottom shows a list of distance readings in centimeters, such as 'Distance: 10', 'Distance: 3', 'Distance: 156', etc. The status bar at the bottom indicates 'Ln 7, Col 23' and 'Arduino Mega or Mega 2560 on /dev/ttyACM0'.

```
ultrasonicSensor.ino
28 duration = pulseIn(echo, HIGH);
29 // Calculate the distance in centimeter (CM):
30 distance = duration * 0.034 / 2;
31
32 // Uncomment this line to return value in IN instead of CM:
33 //distance = distance * 0.3937008
34
35 // Return the distance read from the sensor:
36 return distance;
37 }
38
39 void setup() {
40 // Define inputs and outputs:
41 pinMode(trigger, OUTPUT);
42 pinMode(echo, INPUT);
43
44 // Start the serial monitor:
45 Serial.begin(9600);
46 }
47
48 void loop() {
49 // Print the distance to the serial monitor:
50 Serial.print("Distance: ");
51 Serial.println(getUltrasonicDistance());
52
53 // Wait one second before continuing:
54 delay(1000);
55 }
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on '/dev/ttyACM0')

New Line 9600 baud

Distance: 10
Distance: 3
Distance: 156
Distance: 158
Distance: 3
Distance: 990
Distance: 994
Distance: 991
Distance: 5
Distance: 27
Distance: 167
Distance: 163
Distance: 157
Distance: 153
Distance: 157

Ln 7, Col 23 Arduino Mega or Mega 2560 on /dev/ttyACM0 2

Answer:

--	--

Activity 2.3: Working with sensor - Accelerometer

LSM6DS3 module on the Arduino Nano 33 IoT is an accelerometer and gyroscope sensor.

Hardware Required

Arduino Nano 33 IoT Board (has inbuilt LSM6DS3 module)

USB cable

Software Required

Arduino programming environment

Steps:

Step	Action
1	Write Arduino sketch (or download code from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_accelero.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

```

6
7 // Add Arduino_LSM6DS3 library
8 // from Arduino IDE Library Manager.
9 #include <Arduino_LSM6DS3.h>
10
11 float x, y, z;
12
13 void setup() {
14     Serial.begin(9600); // set baud rate
15     while (!Serial); // wait for port to init
16     Serial.println("Started");
17
18     if (!IMU.begin()) {
19         Serial.println("Failed to initialize IMU!");
20         while (1);
21     }
22
23     Serial.println(
24         "Accelerometer sample rate = "
25         + String(IMU.accelerationSampleRate()) + " Hz");
26 }
27
28 void loop() {
29     // read accelero data
30     if (IMU.accelerationAvailable()) {
31         IMU.readAcceleration(x, y, z);
32     }
33
34     Serial.println(
35         String(x) + ", " + String(y) + ", " + String(z));
36
37     delay(1000);
38 }

```

2

Question: Spec of LSM6DS3 is available here (https://content.arduino.cc/assets/st_imu_lsm6ds3_datasheet.pdf).

Identify at least 3 attributes of this sensor you think important to work with.

Answer:

1) The accelerometer as this can give detailed information about if the device is falling or changing directions. The accelerometer has many real world applications where it would be beneficial such as identifying if a patient has fallen as a sharp change in acceleration would then make that known. Further, an accelerometer could be used to identify if people have crashed their cars when driving this is because the acceleration would be constant and very quickly stop.

2) The gyroscope used in conjunction with the accelerometer will ensure accurate calculations of direction of travel. This is because gyroscopes calculate angular

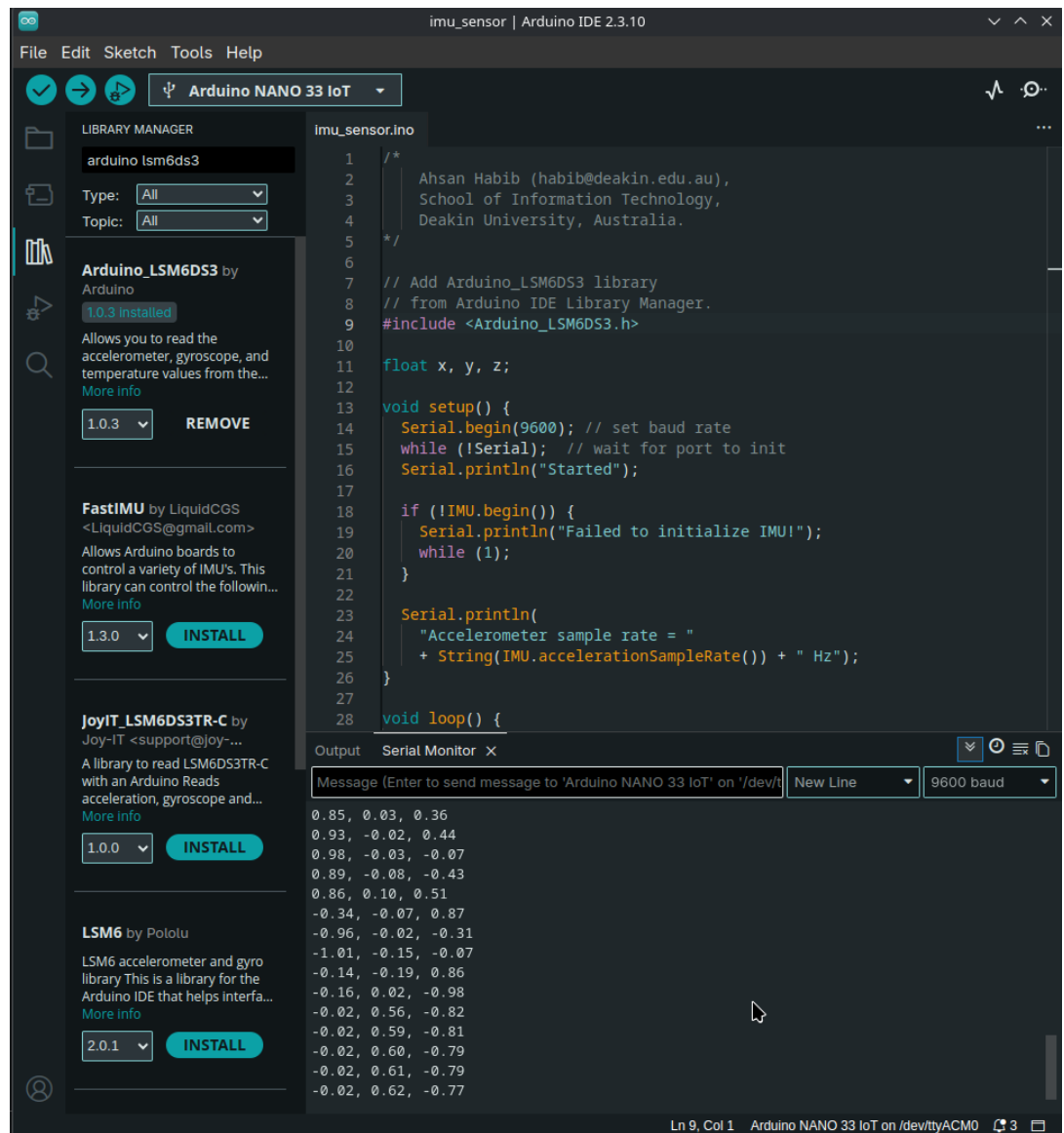
momentum and when combined with the information provided by the accelerometer, direction can be ascertained.

3) The free-fall interrupt which is a builtin feature is very useful as this will allow the device to be used in many contexts where finding out if someone or something is falling. Such as a person falling from height when working at heights, patients falling, or a controller that is controlling heavy machinery has fallen out of the operators hands and the machine must immediately stop functioning.

3

Question: Take a screenshot of your Serial Monitor displaying sensor readings. Add the image here.

Answer:



4	Question: Identify the max sampling rate and consider reducing the delay (line 37 in the sketch) to increase the number of samples. Summarise your findings here. Answer: The max sampling rate of the IMU is as quick as the chip can go i.e. the clock speed of the device. As the delay decreased to 0 the results generated by the device changed almost instantly as the device was rotated in 3D space. Upon checking the documentation for the arduino nano 33 iot, it stated that the device is configured by default with the library at 104hz which means that we can receive a new sample every 9.6 milliseconds. However, the true maximum sampling rate of the LSM6DS3 for the accelerometer is 6 664 Hz and 1 666 Hz for the gyroscope. This means if set to a higher rate we could receive an enormous amount of samples in less than a second. That is at least $1 / 1\,666$ samples a second or a sample every 0.6 milliseconds.

Activity 2.4: Plot data using Python Notebook

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. You can find detail in official website (<https://matplotlib.org>).

Hardware Required

No hardware required

Software Required

Python Jupyter notebook

Steps:

Step	Action
1	Download the Jupyter notebook week2_notebook.ipynb from here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week 2/week2_notebook.ipynb). Follow the instructions in the notebook to carry out instructions and finally convert the notebook to PDF so you can combine it with this activity sheet PDF.